

[복사본]개인용 로컬 프로젝트 관리 앱 설계

1. 개요

- **목표:**

1인 사용자를 위한 **로컬 프로젝트 관리/시각화 데스크톱 앱**

(업무 프로젝트의 개요, 일정(WBS), 변경 이력, 회의록, 개발 정보를 통합 관리 + 시각화)

- **형태:**

- C# 로컬 백엔드 + SQLite/파일

- WPF 데스크톱 앱(WebView2)

- React 기반 웹 UI(시각화 중심)

- **컨셉:**

“옵시디언 + 간트 차트 + 노드 그래프 + 타임라인”을 로컬 전용으로 통합한 개인용 PM 허브

2. 기능 목록

프로젝트 관리

- **프로젝트 개요 관리:**

- 기간, 목표, 예산, 참여 인원, 주요 산출물, 관련 링크

- 프로젝트별 폴더/데이터 자동 생성

- **프로젝트 대시보드:**

- 개요, 주요 마일스톤, 최근 변경 이력, 최근 회의, 개발 정보 요약

일정/WBS 관리

- **작업 관리:**

- 작업명, 담당자, 기간(시작/종료), 상태(예정/진행/완료), 비교

- **간트 차트 시각화:**

- 작업 바, 마일스톤, 기간 스케일, 작업 클릭 시 상세 보기

- **버전 관리:**

- WBS v1, v2 등 버전별 저장

변경 이력 관리

- **변경 이력 CRUD:**

- 날짜, 변경 내용, 영향도, 관련 문서 링크

- **타임라인 시각화:**

- 날짜별 카드 형태, 클릭 시 상세 내용

회의록 관리

- **회의록 작성:**
 - 날짜, 참석자, 주제, 주요 결정사항, 논의 내용, Action Items
- **회의록 목록/검색:**
 - 날짜순, 키워드 검색
- **프로젝트 개요 연동:**
 - 최근 회의 요약 표시

개발 정보 관리

- **DevInfo 항목 타입:**
 - **markdown:** 직접 입력 문서
 - **file:** PPT, Excel, HWP, PDF 등 바이너리 파일
 - **link:** 외부 링크(Notion, Confluence, GitHub 등)
- **파일 처리:**
 - 프로젝트 폴더 내 /DevFiles 저장
 - DB에는 파일 경로/타입만 저장
 - 백엔드에서 OS 기본 앱으로 파일 열기

시각화 대시보드

- **프로젝트 맵(노드 그래프):**
 - 프로젝트 ↔ WBS ↔ 회의록 ↔ 변경 이력 ↔ 개발 정보 노드 연결
- **일정 요약 그래프:**
 - 월별 작업량, 마일스톤 표시
- **변경 이력 히트맵:**
 - 영향도/빈도 기반 색상 표현

로컬/품질 관련

- **로컬 저장:** SQLite + 파일(Markdown/JSON/바이너리)
- **자동 저장:** 입력 시 자동 저장(디바운스)
- **백업/복원:** 프로젝트 폴더 단위 백업/복원
- **오류 처리:** API/파일 오류 표시
- **설정:** 마지막 프로젝트, 창 크기, 테마 등 로컬 설정

3. 사용 언어/프레임워크

- **백엔드:**

- 언어: C#
- 프레임워크: ASP.NET Core (Minimal API 또는 Web API)
- DB: SQLite (EF Core)
- 데스크톱 앱:
 - 언어: C#
 - 프레임워크: WPF + WebView2
- 프론트엔드:
 - 언어: TypeScript/JavaScript
 - 프레임워크: React (+ Vite)
 - 스타일: TailwindCSS
 - 시각화 라이브러리:
 - 간트 차트: ECharts / Frappe Gantt
 - 노드 그래프: Cytoscape.js
 - 타임라인: Vis.js
 - Markdown 렌더링: React Markdown

4. 프로젝트 아키텍처

레이어 구조

- **Core**
 - 도메인 모델(Project, WBS, ChangeLog, Meeting, DevInfo 등)
 - DTOs
 - 인터페이스(IRepository, IService 등)
- **Infrastructure**
 - EF Core(SQLite) 컨텍스트
 - Repository 구현체
 - 파일 저장소(Markdown/JSON/DevFiles)
- **Application**
 - 비즈니스 서비스(ProjectService, WbsService, ChangeLogService, DevInfoService 등)
 - UseCase/Validation/Mapping
- **WebService**
 - ASP.NET Core API

- 엔드포인트/컨트롤러
- DI 구성
- **DesktopApp (WPF)**
 - WebView2 Shell
 - 백엔드 서버 프로세스 실행/종료
 - 로컬 설정 관리

의존성 방향

- Core
- Infrastructure → Core
- Application → Core, Infrastructure
- WebService → Application, Core
- DesktopApp → (직접 참조 없음, WebService EXE를 프로세스로 실행)

실행 흐름

1. DesktopApp 실행
 2. WPF가 WebService(EXE)를 별도 프로세스로 실행
 3. WebView2가 http://localhost:포트로 React UI 로드
 4. React → WebService API 호출
 5. WebService → Application → Infrastructure → DB/파일 처리
 6. 결과를 React UI에 시각화
-

5. 소스 파일 구조 (예시)

```
/solution
/Core
/Domain
Project.cs
WbsItem.cs
ChangeLog.cs
Meeting.cs
DevInfoItem.cs
/DTOs
ProjectDto.cs
...
/Interfaces
IProjectRepository.cs
IWbsService.cs
IDevInfoService.cs
...

/Infrastructure
/Persistence
AppDbContext.cs
ProjectRepository.cs
WbsRepository.cs
...
/FileStorage
MarkdownStorage.cs
DevFilesStorage.cs
/Config
PathResolver.cs

/Application
/Services
ProjectService.cs
WbsService.cs
ChangeLogService.cs
MeetingService.cs
DevInfoService.cs
/Validation
ProjectValidator.cs
...
/Mapping
MappingProfile.cs (AutoMapper 등)

/WebService
Program.cs
/Endpoints
ProjectEndpoints.cs
WbsEndpoints.cs
```

6. 추가로 암시된/내포된 요소

- **확장성:**
 - 나중에 웹 전용 버전, Electron 버전, 모바일 웹도 같은 WebService 재사용 가능
- **테스트 용이성:**
 - Core + Application은 순수 로직이라 단위 테스트에 적합
- **백업 전략:**
 - 프로젝트 폴더 + SQLite 파일 + DevFiles 폴더 통째로 백업
- **보안/범위:**
 - 1인 로컬 사용 전제 → 인증/권한 관리 생략
 - WebService는 localhost 전용